

ESCUELA POLITÉCNICA NACIONAL



Informe técnico de comparación del tiempo de respuesta entre Polling e Interrupciones de Hardware mediante Arduino UNO y ESP32

Arquitectura de Computadores

Nombres:

Armijo Iza Emily Raquel

Gomez Guaman Aitana Raquel

Recalde Amaguayo Ariel Emilio

Curso: 1CD

Ing. Ronie Martínez Msc.

Fecha de entrega: 12/Julio/2026

Contenido

Resumen Ejecutivo	4
1. Introducción	4
2. Objetivos	5
2.1 Objetivo General	6
2.2 Objetivos Específicos	6
3. Marco Teórico	6
3.1 Arquitectura de Computadores	6
3.2 Sistemas Embebidos	7
3.3 Dispositivos de Entrada y Salida	7
3.4 Polling	8
3.5 Interrupciones de Hardware	9
3.6 Tiempo de Respuesta	9
3.7 Arduino UNO	10
3.8 ESP32	10
3.9 Relación con Arquitectura de Computadores	11
4. Materiales Utilizados	11
5. Diseño del Sistema	12
5.1 Sistema A – Polling	12
5.2 Sistema B – Interrupciones	13
6. Descripción del Hardware	13
6.1 Descripción del montaje del Sistema A	14
6.2 Descripción del montaje del Sistema B	15
7. Metodología	15
Etapa 1. Investigación	16
Etapa 2. Diseño del circuito	16
Etapa 3. Desarrollo del software	16
Etapa 4. Implementación	16
Etapa 5. Pruebas	17
8. Diagrama de Flujo	17
Sistema A – Polling	17
9. Desarrollo del Software	18
9.1 Sistema A – Implementación mediante Polling	18
9.2 Funcionamiento del algoritmo Polling	19

9.3 Sistema B – Implementación mediante Interrupciones	19
9.4 Funcionamiento del algoritmo basado en Interrupciones	20
10. Explicación del Código Fuente	21
10.1 Configuración de los pines	21
10.2 Inicialización del sistema	21
10.3 Tarea principal	21
10.4 Detección mediante Polling.....	22
10.5 Detección mediante Interrupciones.....	22
10.6 Medición del tiempo	23
11. Resultados Experimentales	23
12. Análisis de Resultados	24
13. Conclusiones	25
14. Bibliografía	27

Resumen Ejecutivo

El informe compara el tiempo de respuesta y la eficiencia de dos mecanismos de atención de eventos de entrada/salida en sistemas embebidos: Polling (implementado en un Arduino UNO) e Interrupciones de Hardware (implementado en un ESP32). Para la evaluación, ambos microcontroladores ejecutan una tarea principal idéntica que consiste en encender secuencialmente una serie de LEDs con retardos de 150 milisegundos, simulando una carga de procesamiento, mientras deben detectar la pulsación de un botón externo.

En el sistema por Polling, la CPU consulta periódicamente el estado del pulsador mediante la función `digitalRead()`, pero solo lo hace al finalizar el retardo de la tarea principal. Esto provoca que el tiempo de respuesta sea variable y dependa del instante en que se presiona el botón, desperdiciando recursos del procesador. Por el contrario, el sistema basado en interrupciones (`attachInterrupt()`) permite que el hardware notifique automáticamente a la CPU en el momento exacto del evento, pausando la tarea actual para ejecutar una Rutina de Servicio de Interrupción (ISR) y logrando un tiempo de respuesta casi instantáneo y constante.

Los resultados experimentales demuestran que las Interrupciones de Hardware son significativamente más eficientes que el Polling. Liberan a la CPU de la consulta continua de periféricos, optimizan el uso de los recursos del sistema y son el mecanismo adecuado para aplicaciones críticas o de tiempo real donde una latencia mínima es indispensable.

1. Introducción

Dentro de la arquitectura de computadores, uno de los aspectos fundamentales es la comunicación entre el procesador y los dispositivos de entrada y salida (E/S). Sensores, botones, teclados, pantallas y módulos de comunicación generan continuamente eventos que deben ser detectados y atendidos por la CPU.

Existen diversas estrategias para realizar esta comunicación. Dos de las más importantes son el Polling (sondeo) y las interrupciones de hardware. En el primer caso, el procesador consulta constantemente el estado de los dispositivos de entrada hasta detectar un cambio. En el segundo, el propio hardware informa al procesador cuando ocurre un evento, permitiendo que la CPU permanezca dedicada a otras tareas hasta ese momento.

Aunque ambos mecanismos permiten resolver el mismo problema, presentan diferencias importantes en cuanto al uso de recursos, tiempo de respuesta y eficiencia del sistema.

Con el propósito de evidenciar estas diferencias, se desarrolló un sistema comparativo compuesto por dos prototipos físicos. El primero fue implementado utilizando un Arduino UNO bajo una estrategia de Polling, mientras que el segundo utiliza un ESP32 configurado mediante interrupciones de hardware. Ambos sistemas ejecutan la misma tarea principal para garantizar que la comparación se realice bajo condiciones equivalentes.

Finalmente, mediante la medición del tiempo de respuesta ante la pulsación de un botón, fue posible analizar el comportamiento de cada método y relacionarlo con los conceptos estudiados en la asignatura de Arquitectura de Computadores.

2. Objetivos

2.1 Objetivo General

Implementar y comparar dos sistemas embebidos que permitan analizar experimentalmente las diferencias entre el método Polling y las Interrupciones de Hardware mediante la medición de sus tiempos de respuesta.

2.2 Objetivos Específicos

- Diseñar un sistema basado en Polling utilizando un Arduino UNO.
- Implementar un sistema basado en interrupciones utilizando un ESP32.
- Simular una tarea principal que mantenga ocupado al procesador mediante una secuencia de LEDs.
- Registrar el tiempo de respuesta generado por ambos métodos.
- Comparar el desempeño de ambos sistemas.
- Relacionar los resultados obtenidos con los conceptos de Arquitectura de Computadores.

3. Marco Teórico

3.1 Arquitectura de Computadores

La Arquitectura de Computadores estudia la organización interna y el funcionamiento de un sistema computacional. Analiza la forma en que interactúan la Unidad Central de Procesamiento (CPU), la memoria, los buses de comunicación y los dispositivos de entrada y salida para ejecutar instrucciones y procesar información.

El funcionamiento básico de cualquier procesador se basa en el ciclo de instrucción, compuesto por tres etapas principales:

- Captación de la instrucción (Fetch).
- Decodificación (Decode).
- Ejecución (Execute).

Durante este ciclo, el procesador también debe responder a eventos externos generados por los dispositivos conectados al sistema. Dependiendo del mecanismo utilizado para detectar dichos eventos, el rendimiento general del computador puede variar significativamente.

3.2 Sistemas Embebidos

Un sistema embebido es un sistema computacional diseñado para realizar funciones específicas dentro de un dispositivo electrónico. A diferencia de un computador de propósito general, un sistema embebido se encuentra optimizado para ejecutar tareas concretas con recursos limitados.

Los microcontroladores Arduino UNO y ESP32 constituyen ejemplos ampliamente utilizados en el desarrollo de sistemas embebidos debido a su bajo costo, facilidad de programación y disponibilidad de interfaces de entrada y salida digitales.

En este proyecto ambos dispositivos fueron utilizados para demostrar distintos mecanismos de atención de eventos externos.

3.3 Dispositivos de Entrada y Salida

Los dispositivos de entrada permiten que el usuario o el entorno envíe información hacia el procesador. Por otra parte, los dispositivos de salida muestran el resultado del procesamiento realizado por la CPU.

En este proyecto:

Dispositivo de entrada

- Pulsador.

Dispositivos de salida

- Cinco diodos LED.

El pulsador representa un evento externo que debe ser detectado por el procesador mientras éste continúa ejecutando otra tarea.

3.4 Polling

El Polling, también denominado sondeo, consiste en que el procesador consulta periódicamente el estado de un dispositivo para determinar si ocurrió algún cambio.

En este método la CPU ejecuta continuamente una instrucción equivalente a: ¿El botón fue presionado?, Si la respuesta es negativa, continúa ejecutando el programa y vuelve a realizar la misma consulta posteriormente.

Este mecanismo resulta sencillo de implementar, pero obliga al procesador a dedicar parte de su tiempo únicamente a verificar el estado del dispositivo, incluso cuando no existe ningún evento.

Las principales ventajas del Polling son su simplicidad y facilidad de implementación. Sin embargo, presenta importantes desventajas relacionadas con el desperdicio de ciclos de procesamiento y un mayor tiempo de respuesta cuando la CPU se encuentra ocupada ejecutando otras tareas.

3.5 Interrupciones de Hardware

Una interrupción constituye un mecanismo mediante el cual un dispositivo externo informa directamente al procesador que requiere atención inmediata.

Cuando ocurre una interrupción, el procesador interrumpe temporalmente la tarea que estaba ejecutando, guarda su contexto interno, ejecuta una rutina especial denominada **Interrupt Service Routine (ISR)** y posteriormente continúa exactamente desde el punto donde había sido interrumpido.

A diferencia del Polling, el procesador únicamente atiende el evento cuando éste realmente ocurre, permitiendo un mejor aprovechamiento de los recursos del sistema.

Las interrupciones constituyen uno de los mecanismos fundamentales utilizados en sistemas operativos, computadores modernos y sistemas embebidos de tiempo real.

3.6 Tiempo de Respuesta

El tiempo de respuesta representa el intervalo transcurrido entre la ocurrencia de un evento y el instante en que el sistema comienza a atenderlo.

En aplicaciones críticas, como sistemas industriales, dispositivos médicos o control de vehículos, un menor tiempo de respuesta resulta indispensable para garantizar el funcionamiento seguro del sistema.

En este proyecto el tiempo de respuesta corresponde al intervalo comprendido entre la pulsación del botón y el momento en que el microcontrolador detecta dicho evento.

3.7 Arduino UNO

El Arduino UNO es una plataforma basada en el microcontrolador **ATmega328P**, ampliamente utilizada en aplicaciones educativas y de prototipado.

Entre sus principales características se encuentran:

- Frecuencia de operación de 16 MHz.
- 32 KB de memoria Flash.
- 2 KB de memoria SRAM.
- 14 pines digitales.
- 6 entradas analógicas.

En este proyecto fue empleado para implementar el sistema basado en Polling.

3.8 ESP32

El ESP32 es un microcontrolador de alto rendimiento desarrollado por Espressif Systems.

Entre sus principales características destacan:

- Procesador de doble núcleo.
- Frecuencia de hasta 240 MHz.
- WiFi integrado.
- Bluetooth.
- Soporte para múltiples interrupciones de hardware.

Estas características permiten desarrollar aplicaciones de mayor complejidad y obtener tiempos de respuesta considerablemente menores frente a eventos externos.

3.9 Relación con Arquitectura de Computadores

Este proyecto se relaciona directamente con los contenidos estudiados durante la asignatura, especialmente con los temas de dispositivos de entrada y salida, módulos de E/S, interrupciones de hardware y gestión del procesador. Además, permite observar de forma práctica cómo la forma de atender un evento externo influye en el rendimiento general del sistema, aspecto fundamental en el diseño de arquitecturas de computadores.

4. Materiales Utilizados

Para el desarrollo del proyecto fue necesario emplear dos plataformas de desarrollo independientes con el fin de implementar dos mecanismos diferentes para la atención de eventos externos: Polling e Interrupciones de Hardware. Además, se utilizaron componentes electrónicos básicos para construir ambos prototipos sobre una protoboard.

Material	Cantidad	Función
Arduino UNO	1	Implementación del sistema basado en Polling.
ESP32	1	Implementación del sistema basado en interrupciones.
Protoboard	2	Montaje de los circuitos.
LEDs	10	Simulación de la tarea principal mediante una secuencia luminosa.
Resistencias de 220 Ω	10	Protección de los LEDs limitando la corriente.
Pulsadores	2	Generación del evento externo.
Cables Dupont	Varios	Conexión entre los componentes.

Cable USB	2	Alimentación y programación de los microcontroladores.
Arduino IDE	1	Desarrollo y carga del código fuente.

5. Diseño del Sistema

El proyecto fue dividido en dos sistemas independientes con el propósito de realizar una comparación directa entre ambos métodos de atención de eventos.

Cada sistema ejecuta exactamente la misma tarea principal, de manera que la única diferencia entre ellos sea el mecanismo utilizado para detectar la pulsación del botón.

La Figura 1 presenta el diagrama general del funcionamiento del sistema.



Figura 1. Arquitectura general del proyecto.

5.1 Sistema A – Polling

El primer sistema utiliza un Arduino UNO para implementar la técnica de sondeo (Polling). El funcionamiento consiste en ejecutar continuamente una tarea principal

basada en el encendido secuencial de cinco LEDs. Mientras esta tarea se encuentra en ejecución, el programa consulta periódicamente el estado del botón mediante la función `digitalRead()`.

Cuando el botón cambia de estado, el programa registra el instante de detección y calcula el tiempo transcurrido desde la pulsación. Este comportamiento simula el funcionamiento de muchos sistemas sencillos donde la CPU consulta constantemente los dispositivos de entrada.

5.2 Sistema B – Interrupciones

El segundo sistema utiliza un ESP32. En este caso, el botón fue configurado mediante una interrupción de hardware utilizando la función `attachInterrupt()`. Mientras la CPU ejecuta la misma secuencia de LEDs, el hardware permanece vigilando el estado del botón.

Cuando ocurre una pulsación, el controlador de interrupciones detiene momentáneamente la ejecución del programa principal y ejecuta automáticamente la rutina de servicio a la interrupción (ISR). Una vez finalizada dicha rutina, el procesador continúa ejecutando el programa desde el punto donde fue interrumpido. Este mecanismo permite obtener tiempos de respuesta considerablemente menores.

6. Descripción del Hardware

El sistema fue construido utilizando dos montajes físicos independientes. Cada montaje contiene un microcontrolador, un conjunto de LEDs, resistencias limitadoras y un pulsador encargado de generar el evento externo.

Las resistencias fueron utilizadas para limitar la corriente que circula por cada LED, evitando daños tanto en los diodos como en las salidas digitales del microcontrolador. Los LEDs representan visualmente la ejecución de la tarea principal del programa, mientras que el pulsador representa un evento externo que debe ser atendido por el procesador.

6.1 Descripción del montaje del Sistema A

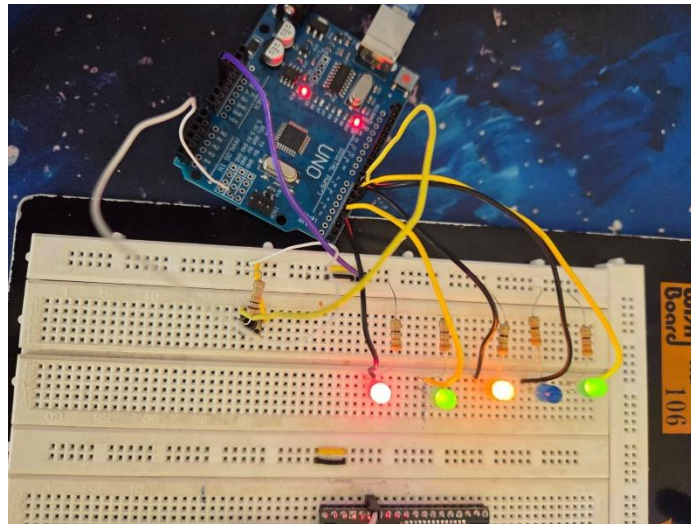


Figura 2. Sistema basado en Polling utilizando Arduino UNO.

En la Figura 2 se observa el circuito correspondiente al Sistema A. El Arduino UNO controla cinco LEDs conectados a través de resistencias de protección. Estos LEDs se encienden de manera secuencial para simular una tarea que mantiene ocupada a la CPU.

En el mismo circuito se encuentra conectado un pulsador que permite generar un evento externo durante la ejecución del programa. El estado de este pulsador es consultado continuamente mediante la técnica de Polling, lo que significa que el microcontrolador verifica repetidamente si el botón ha sido presionado.

Este método implica que el procesador invierte parte de su tiempo consultando el estado del dispositivo incluso cuando no existe ningún cambio.

6.2 Descripción del montaje del Sistema B

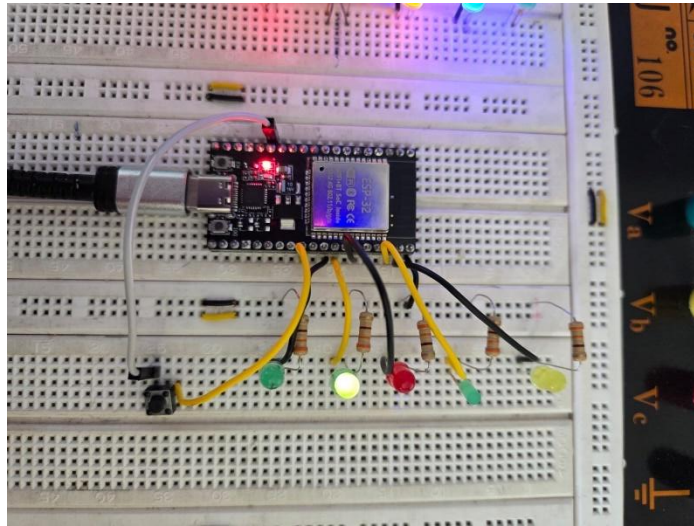


Figura 3. Sistema basado en Interrupciones utilizando ESP32.

La Figura 3 muestra el circuito correspondiente al Sistema B. El ESP32 ejecuta exactamente la misma secuencia de encendido de LEDs que el sistema anterior. Sin embargo, el pulsador se encuentra configurado mediante una interrupción de hardware.

En este caso, el procesador no necesita consultar constantemente el estado del botón, ya que el propio hardware genera una señal de interrupción cuando ocurre la pulsación.

Gracias a este mecanismo, el tiempo de respuesta disminuye considerablemente y el procesador puede dedicar la mayor parte de sus recursos a ejecutar la tarea principal.

7. Metodología

El desarrollo del proyecto se realizó siguiendo las siguientes etapas:

Etapas 1. Investigación

Se revisó la teoría relacionada con la atención de eventos mediante Polling e interrupciones de hardware, así como el funcionamiento interno de los módulos de entrada y salida de los microcontroladores.

Etapas 2. Diseño del circuito

Se diseñaron dos circuitos independientes utilizando protoboards.

Uno de ellos fue construido alrededor de un Arduino UNO y el otro utilizando un ESP32.

Ambos circuitos fueron diseñados para ejecutar exactamente la misma tarea principal.

Etapas 3. Desarrollo del software

Se desarrollaron dos programas independientes.

El primero implementa el método Polling.

El segundo implementa una interrupción de hardware mediante una ISR.

En ambos casos se utilizó el entorno de desarrollo Arduino IDE.

Etapas 4. Implementación

Una vez cargados los programas en los microcontroladores, se verificó el correcto funcionamiento del encendido secuencial de los LEDs y la detección del botón.

Etapa 5. Pruebas

Se realizaron múltiples pulsaciones del botón durante diferentes instantes de la secuencia de LEDs.

Para cada prueba se registró el tiempo de respuesta mostrado en el Monitor Serial.

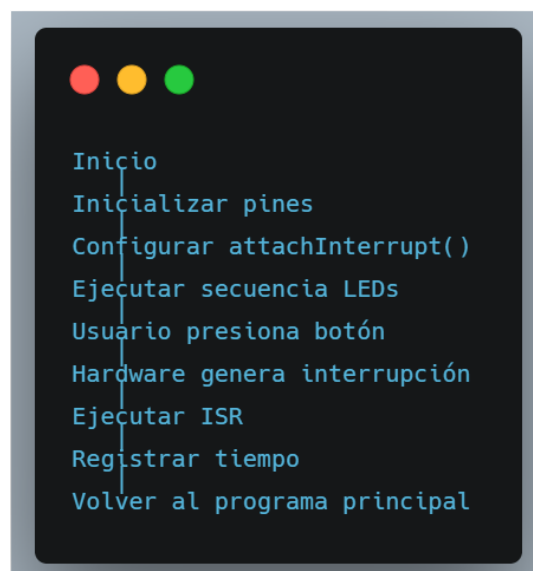
Posteriormente se compararon los resultados obtenidos en ambos sistemas.

8. Diagrama de Flujo

Sistema A – Polling



Sistema B – Interrupción



9. Desarrollo del Software

En esta sección se describe el funcionamiento interno de los programas implementados en ambos microcontroladores. Aunque ambos realizan la misma tarea principal, la forma en que detectan el evento externo es diferente, permitiendo comparar el comportamiento del procesador bajo dos estrategias distintas.

9.1 Sistema A – Implementación mediante Polling

El Sistema A fue desarrollado utilizando un Arduino UNO. El programa tiene como objetivo demostrar el funcionamiento del método Polling mientras el procesador ejecuta una tarea principal.

Al iniciar la ejecución, el programa configura cinco pines digitales como salidas para controlar los LEDs y un pin como entrada con resistencia pull-up interna para conectar el pulsador.

Posteriormente, se habilita una **Pin Change Interrupt (PCINT)** sobre el pin 7. Es importante destacar que esta interrupción **no se utiliza para atender el botón**, sino únicamente para registrar el instante exacto en que fue presionado y así disponer de una referencia de tiempo para medir el retraso del Polling.

La tarea principal consiste en encender secuencialmente cinco LEDs, manteniendo cada uno activo durante 150 milisegundos. Este retardo simula una carga de trabajo para el procesador.

Finalizado cada retardo, el programa consulta el estado del pulsador mediante la función `digitalRead()`. Si detecta que el botón fue presionado, calcula la diferencia entre el instante

registrado por la interrupción de referencia y el momento en que el Polling finalmente detectó el evento. Ese valor corresponde al tiempo de respuesta del sistema.

Finalmente, el resultado es mostrado en el Monitor Serial y la secuencia vuelve a comenzar.

9.2 Funcionamiento del algoritmo Polling

El funcionamiento puede resumirse de la siguiente manera:

1. Inicializar los pines del sistema.
2. Encender un LED.
3. Esperar 150 ms.
4. Leer el estado del botón.
5. Si el botón no fue presionado, continuar con el siguiente LED.
6. Si fue presionado, calcular el retardo y mostrar el resultado.

Durante el tiempo que dura el delay(150), el procesador permanece ocupado y no verifica el estado del botón. Como consecuencia, la respuesta depende del instante exacto en que el usuario realizó la pulsación.

9.3 Sistema B – Implementación mediante Interrupciones

El Sistema B fue implementado utilizando un ESP32.

Al igual que el sistema anterior, se configuran cinco pines digitales como salidas para controlar los LEDs y un pin de entrada para el pulsador.

La principal diferencia consiste en que el botón es configurado mediante la función `attachInterrupt()`, asociando la señal del pulsador con la rutina de servicio denominada `ISR_boton()`.

Mientras el programa principal ejecuta la secuencia de LEDs, el hardware permanece supervisando continuamente el estado del botón sin necesidad de que la CPU lo consulte.

Cuando ocurre una pulsación, el controlador de interrupciones detiene momentáneamente la ejecución del programa principal y ejecuta inmediatamente la rutina de interrupción.

Dentro de dicha rutina se apagan todos los LEDs y se mide el tiempo necesario para realizar esta acción. Posteriormente, el programa principal continúa exactamente desde el punto donde fue interrumpido.

9.4 Funcionamiento del algoritmo basado en Interrupciones

El funcionamiento del sistema puede resumirse mediante la siguiente secuencia:

1. Inicializar los pines.
2. Configurar la interrupción mediante `attachInterrupt()`.
3. Ejecutar la secuencia de LEDs.
4. Esperar una interrupción.
5. Cuando ocurre la pulsación, ejecutar la ISR.
6. Apagar inmediatamente todos los LEDs.
7. Registrar el tiempo de respuesta.
8. Continuar con el programa principal.

En este caso, el procesador no necesita consultar continuamente el estado del botón, ya que el propio hardware genera la señal de interrupción cuando detecta el evento.

10. Explicación del Código Fuente

10.1 Configuración de los pines

En ambos programas se definen inicialmente los pines correspondientes a los LEDs y al pulsador.

El arreglo LED_PINS[] almacena los pines utilizados para controlar la secuencia luminosa.

El botón es configurado como entrada utilizando la resistencia interna INPUT_PULLUP, lo que garantiza un estado lógico estable cuando el pulsador permanece sin presionar.

10.2 Inicialización del sistema

En la función setup() se realizan las siguientes acciones:

- Inicialización de la comunicación serial.
- Configuración de todos los pines.
- Inicialización del mecanismo de detección correspondiente (Polling o Interrupción).
- Presentación de mensajes informativos en el Monitor Serial.

Esta etapa únicamente se ejecuta una vez al iniciar el programa.

10.3 Tarea principal

Ambos programas ejecutan exactamente la misma tarea principal.

Esta consiste en:

- apagar todos los LEDs,
- encender un único LED,
- esperar 150 ms,
- pasar al siguiente LED.

El retardo de 150 milisegundos representa una carga de procesamiento que mantiene ocupada a la CPU.

Desde el punto de vista de la Arquitectura de Computadores, este retardo simula que el procesador está realizando operaciones que requieren tiempo de ejecución.

10.4 Detección mediante Polling

En el Sistema A, la detección del botón se realiza utilizando la función `digitalRead()`.

Esta instrucción consulta el estado eléctrico del pin correspondiente al pulsador.

Sin embargo, dicha consulta solo ocurre después de finalizar el retardo de 150 ms.

Como consecuencia, el tiempo de respuesta depende del instante en que el programa vuelve a ejecutar la lectura del botón.

10.5 Detección mediante Interrupciones

En el Sistema B, la función `attachInterrupt()` configura el hardware del ESP32 para detectar automáticamente una transición de nivel en el pin del botón.

Cuando ocurre dicha transición, el microcontrolador suspende inmediatamente la ejecución del programa principal y ejecuta la rutina `ISR_boton()`.

Dentro de esta rutina se apagan todos los LEDs antes de que el programa principal continúe ejecutándose.

Este comportamiento constituye la principal ventaja de las interrupciones frente al Polling.

10.6 Medición del tiempo

En ambos programas se emplea la función `micros()`, la cual devuelve el tiempo transcurrido desde el inicio de la ejecución del microcontrolador expresado en microsegundos.

En el Sistema A se calcula la diferencia entre el instante real de la pulsación, registrado mediante la interrupción PCINT de referencia, y el momento en que el Polling detecta el botón. Ese valor representa el retraso introducido por el sondeo.

En el Sistema B, la medición se realiza directamente dentro de la rutina de interrupción, calculando el tiempo que tarda el procesador en reaccionar desde que recibe la interrupción hasta completar la acción de apagar los LEDs.

11. Resultados Experimentales

En ambos sistemas se realizaron múltiples pruebas presionando el pulsador durante diferentes momentos de la secuencia de LEDs.

Los resultados mostraron que el tiempo de respuesta del sistema basado en Polling varía considerablemente dependiendo del instante en que el usuario realiza la pulsación. Esto ocurre porque el programa únicamente consulta el estado del botón al finalizar cada ciclo de la tarea principal.

Por el contrario, el sistema basado en interrupciones presentó tiempos de respuesta prácticamente constantes y muy reducidos, ya que la CPU fue notificada inmediatamente por el hardware sin necesidad de esperar a que terminara la tarea en ejecución.

12. Análisis de Resultados

El objetivo principal del proyecto fue comparar el comportamiento de dos mecanismos utilizados para atender eventos externos en sistemas embebidos: el método de Polling y las Interrupciones de Hardware. Aunque ambos sistemas ejecutaron la misma tarea principal y utilizaron componentes similares, el mecanismo empleado para detectar la pulsación del botón produjo diferencias significativas en el tiempo de respuesta.

En el sistema implementado mediante Polling, el microcontrolador debía consultar periódicamente el estado del pulsador utilizando la función `digitalRead()`. Esta consulta únicamente se realizaba después de finalizar cada ciclo de la tarea principal, la cual consistía en el encendido secuencial de los LEDs acompañado de un retardo de 150 ms. Como consecuencia, si el usuario presionaba el botón mientras el procesador se encontraba ejecutando el retardo, el evento no era detectado de forma inmediata, sino únicamente cuando el programa volvía a consultar el estado del pin.

Por el contrario, en el sistema basado en interrupciones el botón fue configurado utilizando una interrupción de hardware. En este caso, el hardware notificó automáticamente al procesador cuando ocurrió la pulsación, provocando que la CPU suspendiera temporalmente la tarea principal para ejecutar la rutina de servicio a la interrupción (ISR). Una vez atendido el evento, el programa continuó su ejecución normal.

Esta diferencia demuestra que las interrupciones permiten aprovechar mejor los recursos del procesador, ya que la CPU no necesita dedicar tiempo a consultar continuamente el estado de los dispositivos de entrada. En aplicaciones donde la velocidad de respuesta es crítica, como sistemas de control industrial, robótica o dispositivos médicos, este mecanismo resulta significativamente más eficiente.

Desde el punto de vista de la Arquitectura de Computadores, el proyecto permitió observar cómo la forma en que el procesador administra los dispositivos de entrada y salida influye directamente sobre el rendimiento general del sistema.

Tabla 2. Comparación entre ambos métodos

Característica	Polling	Interrupciones
Detección del evento	Consulta periódica	Notificación automática
Uso de CPU	Alto	Bajo
Tiempo de respuesta	Variable	Muy bajo
Complejidad de implementación	Baja	Media
Eficiencia	Menor	Mayor
Aplicaciones recomendadas	Sistemas simples	Sistemas de tiempo real

Tabla 3. Tabla comparativa

Métrica	Sistema A <i>Polling</i>	Sistema B <i>Interrupciones</i>
Retardo mínimo	0.236 ms	0.003 ms
Retardo máximo	143.592 ms	0.004 ms

Retardo promedio	= 63.44 ms	= 0.036 ms
Consistencia	Muy variable (depende del momento de la secuencia)	Prácticamente constante
Orden de magnitud	Milisegundos	Microsegundos

El sistema B respondió, en promedio, más de 17,000 veces mas rápido que el sistema A (63.44 ms vs. 0.0036 ms). Esta diferencia demuestra de forma cuantitativa la ventaja de las interrupciones del hardware frente al sondeo cuando el tiempo de respuesta es crítico.

13. Conclusiones

Se logró implementar satisfactoriamente dos sistemas embebidos que utilizan diferentes mecanismos para atender eventos externos, permitiendo comparar experimentalmente el método Polling y las Interrupciones de Hardware.

Los resultados obtenidos demostraron que el método Polling introduce un retardo adicional debido a que el procesador únicamente verifica el estado del pulsador cuando finaliza la tarea que se encuentra ejecutando. En consecuencia, el tiempo de respuesta depende del instante en que ocurre la pulsación.

El sistema basado en interrupciones presentó tiempos de respuesta significativamente menores, ya que el hardware notifica inmediatamente al procesador la ocurrencia del evento, permitiendo atenderlo sin necesidad de realizar consultas periódicas.

Desde el punto de vista de la Arquitectura de Computadores, el proyecto permitió comprender la importancia de los mecanismos de entrada y salida y su influencia sobre el rendimiento general del sistema.

La experiencia práctica confirmó los conceptos estudiados en clase relacionados con dispositivos de entrada y salida, módulos de interrupción y gestión del procesador, demostrando que las interrupciones constituyen la alternativa más eficiente para aplicaciones donde el tiempo de respuesta representa un requisito fundamental.

14. Bibliografía

[1] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design RISC-V Edition: The Hardware Software Interface*, 2nd ed. Burlington, MA, USA: Morgan Kaufmann, 2021.

[2] W. Stallings, *Computer Organization and Architecture: Designing for Performance*, 11th ed. New York, NY, USA: Pearson, 2022.

[3] Arduino, "Arduino Documentation," Arduino, 2025. [Online]. Available: <https://docs.arduino.cc>. [Accessed: 11-Jul-2026].

[4] Espressif Systems, "ESP32 Technical Reference Manual," Espressif Systems, Shanghai, China, Version 5.5, 2024.

[5] Espressif Systems, "ESP-IDF Programming Guide," Espressif Systems. [Online]. Available: <https://docs.espressif.com/projects/esp-idf/>. [Accessed: 11-Jul-2026].

[6] Microchip Technology Inc., *ATmega328P Datasheet*, Chandler, AZ, USA, 2018.

[7] Microchip Technology Inc., *AVR Instruction Set Manual*, Rev. DS40002198C,
Chandler, AZ, USA, 2024.

[8] B. Razavi, *Fundamentals of Microelectronics*, 3rd ed. Hoboken, NJ, USA: John
Wiley & Sons, 2021.